

OmniMorse — where things stand

Store-readiness status · 11 September 2026 · version 0.3.0 (build 6), unbumped and untagged on purpose

951	9/9	14	0
unit & property tests	E2E, both platforms	screenshots, both stores	usable video files

Done

Item	State
Six tester-reported defects from 0.2.1 — Android vibration, torch flicker, the dead Speak button, the Tap button covering the screen, the carousel, the version label	shipped in 0.3.0
App renamed OmniMorse, punchline <i>Encode. Decode. Learn.</i> — app, store copy and the portfolio site	done
Default message SOS, input focused on open, low-volume toast at ≤30%	done
Privacy policy — live, verified byte for byte, two rename leftovers corrected	published
Support page — answers the torch, vibration, volume, speech and tap questions	published
Play Console — package registered, Data safety, store settings, three tags, listing copy in three languages	filled
App Store Connect copy — subtitle, keywords, promotional text in three languages, plus the answers that are not per language	written
Android screenshots — seven at 1080×2400	ready
iOS screenshots — seven at 1320×2868	ready
Icons, splash, feature graphic — generated from one script	done
CI — Android build extracted to a shared action; screenshots workflow lost 120 duplicated lines	done

Not done

Item	State
The two videos — Play promo and the LinkedIn four-up. Flows, composer, workflow and 47 tests all exist and are proven; no run has yet produced usable footage. One was dispatched as this session ended.	in flight
Play internal testing rollout — morse-0.3.0-6.aab is built and not yet rolled out	pending
Confirm on the Poco that vibration works and the torch stopped blinking	unverified
iPad — supportsTablet is true, so Apple requires a 13-inch screenshot set no workflow produces	decision
The privacy manifest declares CrashData alone while Play declares Device IDs too	to verify
Firebase Analytics — four things move together when it ships, including the advertising-ID answer	deferred

Dependabot — transitive advisories needing `pnpm.overrides`

deferred

v0.3.0 tag and version bump — deliberately held: both fire workflows that need EAS credits

after 1 Oct

EAS builds are blocked until 1 October. The credit pool is shared across platforms and ran out on 9 September. `--local` is free and unaffected, and `eas submit` never needed credits — there is no reason to rebuild in order to submit.

The videos, in detail

This is the one unfinished deliverable, and the only one where the machinery is complete but the output is not. It is worth reading before the next attempt.

Three runs reported success and produced unusable footage. `continue-on-error` on the emulator step is what preserves partial footage when a flow falls over — and it also swallows five flows failing in a row. **Verification is looking at a frame**, not at the run's conclusion. Nothing else caught any of the three failures below.

Run	Symptom	Cause, and the fix
1	no clips at all	<code>android-emulator-runner</code> splits its <code>script:</code> on newlines and runs each line through its own <code>sh -c</code> , so a shell function was torn apart mid-brace. The logic moved to a script file.
2	"Pixel Launcher isn't responding" across every frame	A <code>pixel_6</code> profile at 1080p through swiftshader, with animations on for the whole job, ANRs the launcher; the dialog then hid the app from Maestro. Error dialogs are suppressed, and animations moved from the workflow to the recorder — boot cheap, record properly.
3	the wrong screen in three of four cells	<code>waitForAnimationToEnd: timeout: N</code> does not wait for N — N is a maximum, and returns in under a second on a screen that is not animating. The dwell now comes from the recorder, which runs a fixed length and is never stopped early.

Two design decisions that came out of it

- **Both outputs trim from the END of a clip, never the front.** Everything variable — boot, Maestro's driver, the app launch, flow retries — happens at the front; what is worth watching is at the end. A fixed front trim was wrong by about fifty seconds the moment flows retried.
- **Playback drops to 5 WPM before signalling.** A message at the default speed is over in about three seconds, which is why the play assertion failed — and a cell that plays for three seconds of sixteen is a still image for the rest. Duration is bought with speed rather than with filler characters.

When a run finally produces good footage

- Re-framing costs seconds of `ffmpeg`, not another run: re-compose from the `store-videos-raw-clips` artifact.
- **Play takes a YouTube URL for the promo video, not an upload** — and there are five ways to paste one it rejects: extra parameters, private, age-restricted, monetised, or no feature graphic for it to sit on.
- **There is no sound and there cannot be.** `screenrecord` captures video only. Music belongs on the YouTube upload.

Where to read more

File	What it holds
<code>docs/NEXT-STEPS.md</code>	the live handover — start here
<code>docs/VIDEO.md</code>	the video pipeline and every trap in it
<code>docs/PLAY-CONSOLE.md</code>	every Play answer, and the two that were wrong once

docs/APP-STORE.md	Apple's fields, age rating, review notes, export compliance
docs/MACHINE-SETUP.md	reproducing this Mac: toolchain, credentials, quotas
docs/store-listing/	paste-ready copy, per language, for both stores